

ECE 759
High Performance Computing for Engineering Applications
Assignment 7
Due Friday 05/02/2025 at 23:59 PM

Submit responses to all tasks which don't specify a file name to Canvas in a file called assignment7.{txt, docx, pdf, rtf, odt} (choose one of these formats). Submit all plots (if any) on Canvas. Do not zip your Canvas submission.

All *source files* should be submitted in the **HW07** subdirectory on the **main** branch of your **GitLab** repo. For this assignment, your **HW07** folder should contain **task1.cu**, **task2.cu**, **reduce.cu**, and **matmul.cu**.

All commands or code must work on *Euler* with only the **nvidia/cuda** module loaded. The commands may behave differently on your home/office computer, so be sure to test on *Euler* before you submit. Loading the module is done via

```
$ module load nvidia/cuda
```

Please submit clean code. Consider using a formatter like **clang-format**.

IMPORTANT: Before you begin, copy the provided files from **Assignments/HW07** directory of the **ECE 759 Resource Repo**.

The GitHub link to your code folder should be: <https://github.com/saanna225/repo759/tree/main/HW07>

The answers to the questions are written below in respective place

1. Implement in a file called **matmul.cu** the template functions **matmul_*** as used and described in **matmul.cuh**. Be sure to follow the use of shared memory tiles. These should be based on the tiled matrix multiplication method presented in the class. Your implementation should work for arbitrary matrix dimension $n \leq 2^{14}$ (and **n** is not necessarily a power of 2). More specifics:
 - a) Write a test program **task1.cu** which does the following.
 - For each of the test functions defined in **matmul.cuh**: **matmul_1**, **matmul_2**, and **matmul_3**.
 - Creates and fills however you like row-major representations of $n \times n$ matrices **A**, **B**, and **C**, where **n** is the first command line argument as below.
 - Copies them to the device memory.
 - Calls the functions **matmul_1**, **matmul_2**, and **matmul_3** to produce **C** as the matrix product **AB**.
 - Prints the first element of the resulting **C**.
 - Prints the last element of the resulting **C**.
 - Prints the time taken to run the matrix multiplication in *milliseconds* using CUDA events.
 - Compile: **nvcc task1.cu matmul.cu -Xcompiler -O3 -Xcompiler -Wall -Xptxas -O3 -std c++17 -o task1**
 - Run (where **n** and **block dim** are positive integers and **n** is not necessarily a multiple of **block dim**; make sure you use **Slurm**): **./task1 n block_dim**
 - Exampled expected output:
1025
561
10256.2
1025.1
561.3
10256.2
1025.1
561.3
60256.2
 - b) On *Euler* using **Slurm**, run **task1** for each value $n = 2^5, 2^6, \dots, 2^{14}$ and generate a plot of the time taken by your algorithm as a function of **n**.

- c) What is the best performing value of `block_dim` when $n = 2^{14}$?
- d) does the performance change depending on the type of the data (i.e. `int`, `float`, `double`)? Why do you think that is?
- e) Present the best runtime for $n = 2^{14}$ from your HW06 matrix multiplication task. Explain why one of them (tilted vs. naive) performs better than the other.
- f) Present the runtime for $n = 2^{14}$ from HW02 (serial implementation `mmul1`) (or state that it goes beyond 10 minutes). Compare the performance between CPU and GPU implementations and explain why one of them is better. It is preferred that you use your own implementation for this comparison, but for those who dropped HW02, simply predict the better approach and explain why it is superior.

ANSWER: c) For full utilization of memory then `block_dim = 32` is a good match giving 1024 threads in a block. Fewer threads leads to under utilization of the GPU giving less optimization .

d) In the plot pdf, we can see that double precision takes much more time to execute since it occupies double the bandwidth of int and float. Float and int datatype follow almost similar pattern across the program execution.

e) By splitting huge matrices into smaller sub-matrices (tiles), each thread block operates on a single tile, tiling enhances performance. Compared to global memory, shared memory loads these tiles far more quickly. Because matrix multiplication necessitates repeatedly accessing rows and columns, this minimizes the quantity of sluggish global memory accesses. The naïve technique, on the other hand, has a significantly larger latency because it continuously retrieves elements from global memory. By facilitating quicker, targeted memory accesses via shared memory, the tiled technique maximizes this and improves overall speed.

f) HW02: Took over 6.78 minutes.

HW06: Approx 56 sec

HW07: 18 sec approx

One core manages all calculations across three nested loops, each of which iterates over n elements, in the CPU version's single-threaded serial implementation.

The GPU version, however, distributes the burden over the X and Y dimensions by utilizing tremendous parallelism. Each thread completes fewer iterations in the tiled version, and access to reused data is further accelerated by the use of fast shared memory. This considerably lowers the overall execution time and lessens the computational load on each thread.

2. Implement in a file called `reduce.cu` the functions `reduce` and `reduce_kernel` as declared and described in `reduce.cuh`. Your `reduce_kernel` should use the alteration from Reduction #4 ("First Add During Load," see lecture slides). The `reduce_kernel` function should be called inside the `reduce` function (repeatedly if needed) until the final sum of the entire array is obtained. To this end:

- a) Write a test program `task2.cu` which will complete the following (some memory management steps are omitted for brevity, but you should implement them in your code):
 - Create and fill an array of length `N` with random numbers in the range `[-1,1]` on the host, where `N` is the first command line argument as below.
 - Copy this host array to device as the `input` array where the reduction will be performed on.
 - Create another `output` array on the device that has its length equal to the number of blocks required for the first call to the kernel function `reduce_kernel`.
 - Call your `reduce` function to sum all the elements in the `input` array, with the `threads_per_block` read from the second command line argument as below.
 - Print the resulting sum.

- Print the time taken to run the `reduce` function in *milliseconds*.
 - Compile: `nvcc task2.cu reduce.cu -Xcompiler -O3 -Xcompiler -Wall -Xptxas -O3 -std c++17 -o task2`
 - Run (where $N \leq 2^{30}$ and `threads_per_block` are positive integers, and `N` is not necessarily a power of 2; make sure you use `Slurm`):
`./task2 N threads_per_block`
 - Example expected output:
102536
3.215
- b) On *Euler* using `Slurm`, run `task2` for each value $n = 2^{10}, 2^{11}, \dots, 2^{30}$ and generate a plot named `task2.pdf` with the time taken by your algorithm as a function of `N` when `threads_per_block` = 1024. Overlay another plot which plots the same relationship with a different choice of `threads_per_block`.